

Assignment: Backend Unit Testing Integration

Course: CSE 470

Topic: Software Quality Assurance - Unit Testing

Deadline: 08 September , 2026 (11:59 PM)

Reliable software requires rigorous verification. You are required to integrate a unit testing framework into your existing CSE 470 project backend. Your objective is to validate the logic and response handling of your API endpoints using an automated testing suite appropriate for your technology stack (e.g., Jest for Node.js, Pytest for Python/FastAPI, Pest/PHPUnit for PHP).

2. Directory & Naming Standards

To maintain repository hygiene, strict adherence to the following structure is required. Deviations will complicate grading.

- **Location:** All tests must reside in a **tests** directory within your backend root or **src** folder.
- **File Naming Convention:** **{Student_ID}_{Feature_Name}.test.{extension}**

Example Structure:

backend/

```
|— src/
    |— tests/
        |— 19101010_auth_system.test.ts
        |— 21201202_product_catalog.test.js
        |— 20301011_payment_gateway.test.py
```

3. Functional Requirements

1. **Individual Contribution:** Each team member must implement tests for **one specific feature** they developed.
2. **Coverage Scope:** You must write test cases for **all API endpoints** associated with that selected feature.
 - *Minimum Expectation:* If your feature is "User Management," you must test Create, Read, Update, and Delete (CRUD) operations.

3. Authentication Handling:

- **Constraint:** Hardcoded access tokens or credentials are **strictly prohibited**.
- **Requirement:** Your test suite must handle authentication programmatically (e.g., logging in a mock user in a `beforeAll` hook to generate a fresh token for the session).

4. Demo Test Cases (Reference)

Below is a logical breakdown of the test cases you should implement. These examples assume a **"Blog Post Management"** feature. You should adapt this logic to your specific feature.

Case A: Positive Flow (Happy Path)

- **Test 1: Create Resource**
 - *Input:* Valid payload (Title, Content, AuthorID).
 - *Expected Output:* Status 201 Created, returns the created object with an ID.
- **Test 2: Retrieve Resource**
 - *Input:* The ID generated in Test 1.
 - *Expected Output:* Status 200 OK, returns matching data.

Case B: Negative Flow (Error Handling)

- **Test 3: Validation Error**
 - *Input:* Payload missing a required field (e.g., missing "Title").
 - *Expected Output:* Status 400 Bad Request, returns specific validation error message.
- **Test 4: Resource Not Found**
 - *Input:* Request to Update/Delete a generic ID that does not exist (e.g., ID: 999999).
 - *Expected Output:* Status 404 Not Found.

Case C: Security & Boundary

- **Test 5: Unauthorized Access**
 - *Input:* Valid request but without an Authorization header/Token.
 - *Expected Output:* Status 401 Unauthorized or 403 Forbidden.

5. Code Structure Example (Jest/Node.js)

```
describe('Feature: Blog Management (ID: 10101010)', () => {  
  let authToken = "";  
  
  // PRE-CONDITION: Dynamic Auth  
  beforeAll(async () => {  
    const response = await request(app)  
      .post('/api/auth/login')
```

```
    .send({ email: 'test@user.com', password: 'password123' });  
    authToken = response.body.token;  
  });
```

// TEST 1: Create

```
it('should create a new blog post', async () => {  
  const res = await request(app)  
    .post('/api/blogs')  
    .set('Authorization', `Bearer ${authToken}`)  
    .send({ title: 'Unit Testing', content: 'Content here...' });
```

```
  expect(res.statusCode).toEqual(201);  
  expect(res.body).toHaveProperty('id');  
});
```

// TEST 2: Validation Failure

```
it('should return 400 if title is missing', async () => {  
  const res = await request(app)  
    .post('/api/blogs')  
    .set('Authorization', `Bearer ${authToken}`)  
    .send({ content: 'Missing title' });
```

```
  expect(res.statusCode).toEqual(400);  
});  
});
```

Extra Resources: <https://github.com/TheDeadcoder/nodejs-init>

6. Submission

- **Deliverables:**
 1. The source code of your test file.
 2. A screenshot of your terminal showing the test suite passing (green ticks).
 3. Put these in a zip file renamed by your student id and submit.
- **Format:** *{Student_ID}.zip*